

Análise de Operações Totais vs N

Quicksort e Shell Sort

Docente: Prof. Dr. Diego Gervasio Frias Suarez
Discente: Guilherme Dos Santos Modesto Teixeira

1 Visão Geral

Este projeto analisa o número total de operações (comparações + trocas/atribuições) realizadas pelos algoritmos Quicksort (iterativo com pivô mediana-de-três) e Shell Sort (com sequência de Ciura) em função do tamanho da entrada N. Diferentemente da abordagem tradicional de medir tempo de execução, aqui contamos cada operação elementar, eliminando ruído do ambiente e obtendo curvas puramente algorítmicas.

Também comparamos a curva de operações com a curva de tempo real de execução (ambas normalizadas) para validar que a contagem de operações é um preditor fiel do comportamento observado na prática.

2 Metodologia

```
1 Para cada N em [1k, 2k, 5k, 10k, 20k, 50k, 100k]:  
2   1. Gera 20 listas aleatorias de tamanho N  
3   2. Para cada lista:  
4       a. Ordena com o algoritmo e conta operacoes  
5       b. Ordena outra copia e mede o tempo real  
6   3. Operacoes medias = media das 20 execucoes  
7   4. Tempo medio = media dos 20 tempos
```

2.1 O que conta como operação?

Algoritmo	Operações contadas
Quicksort	Comparações entre elementos (<, >), trocas (swap) e comparações de índice
Shell Sort	Comparações entre elementos (>), atribuições (arr[j] = ...) e verificação de índice (j >= gap)

3 Explicação do Código

3.1 Quicksort

O Quicksort implementado é iterativo (usa pilha explícita) com escolha de pivô mediana-de-três para evitar o pior caso $O(n^2)$:

```
1 def quicksort(arr):  
2     stack = [(0, len(arr) - 1)]           # Pilha com intervalos [lo, hi]  
3     while stack:  
4         lo, hi = stack.pop()              # Remove um intervalo da pilha  
5         if lo >= hi:                      # Intervalo invalido  
6             continue  
7  
8         # --- Mediana de tres: lo, mid, hi ---  
9         mid = (lo + hi) // 2  
10        if arr[lo] > arr[mid]:             # 1a comparacao
```

```

11         arr[lo], arr[mid] = arr[mid], arr[lo] # swap
12     if arr[lo] > arr[hi]: # 2a comparacao
13         arr[lo], arr[hi] = arr[hi], arr[lo] # swap
14     if arr[mid] > arr[hi]: # 3a comparacao
15         arr[mid], arr[hi] = arr[hi], arr[mid] # swap
16     pivot = arr[mid]
17     arr[mid], arr[hi - 1] = arr[hi - 1], arr[mid] # esconde pivo
18
19     # --- Particionamento de Hoare ---
20     i, j = lo, hi - 1
21     while True:
22         i += 1
23         while arr[i] < pivot: # busca elemento > pivo a esquerda
24             i += 1
25         j -= 1
26         while arr[j] > pivot: # busca elemento < pivo a direita
27             j -= 1
28         if i >= j: # ponteiros se cruzaram
29             break
30         arr[i], arr[j] = arr[j], arr[i] # troca os elementos
31
32     arr[i], arr[hi - 1] = arr[hi - 1], arr[i] # recoloca o pivo
33
34     # --- Empilha sub-intervalos ---
35     stack.append((lo, i - 1)) # sublista esquerda
36     stack.append((i + 1, hi)) # sublista direita
37     return arr

```

Partes principais:

Trecho	Papel
<code>stack = [(0, len(arr)-1)]</code>	Pilha que substitui a recursão; cada entrada é um par (início, fim)
Mediana de três	Escolhe o pivô como a mediana entre <code>arr[lo]</code> , <code>arr[mid]</code> e <code>arr[hi]</code> , garantindo boa partição mesmo em listas ordenadas
<code>while arr[i] < pivot: i += 1</code>	Varre da esquerda até achar um elemento $\geq$ pivô
<code>while arr[j] > pivot: j -= 1</code>	Varre da direita até achar um elemento $\leq$ pivô
<code>if i >= j: break</code>	Se os ponteiros se cruzaram, a partição terminou
<code>stack.append(...)</code>	Em vez de chamar recursivamente, empilha os subintervalos para processar depois

3.1.1 Contagem de operações no Quicksort

A função `quicksort_ops` duplica a lógica acima incrementando um contador `ops` a cada comparação ou swap:

```

1 def quicksort_ops(arr):
2     ops = 0
3     stack = [(0, len(arr) - 1)]
4     while stack:
5         lo, hi = stack.pop()
6         ops += 1 # comparacao lo >= hi
7         if lo >= hi:
8             continue
9         mid = (lo + hi) // 2
10        ops += 3 # 3 comparacoes da mediana
11        if arr[lo] > arr[mid]:
12            arr[lo], arr[mid] = arr[mid], arr[lo]
13            ops += 1 # swap
14        ... # demais comparacoes e swaps
15    return arr, ops

```

3.2 Shell Sort

O Shell Sort generaliza o insertion sort usando gaps decrescentes. A sequência adotada é a de Ciura (1, 4, 10, 23, 57, 132, 301, 701):

```
1 def shell_sort(arr):
2     n = len(arr)
3     gaps = [1, 4, 10, 23, 57, 132, 301, 701]
4     gaps = [g for g in gaps if g < n]           # remove gaps maiores que n
5
6     for gap in reversed(gaps):                  # do maior gap para o menor
7         for i in range(gap, n):                # insertion sort com passo = gap
8             temp = arr[i]
9             j = i
10            while j >= gap and arr[j - gap] > temp: # desloca elementos
11                arr[j] = arr[j - gap]
12                j -= gap
13            arr[j] = temp                        # insere no lugar correto
14    return arr
```

Partes principais:

Trecho	Papel
gaps = [1, 4, 10, 23, 57, 132, 301, 701]	Sequência de Ciura — gaps com crescimento $\sim 2.2\times$, empiricamente eficiente
reversed(gaps)	Itera do maior gap para o 1, refinando a ordenação a cada passo
for i in range(gap, n)	Insertion sort espacial: cada elemento i é comparado com o que está gap posições atrás
while j >= gap and arr[j - gap] > temp	Desloca para a direita enquanto o elemento anterior (no gap) for maior
arr[j] = temp	Insere o elemento na posição correta dentro da sub-sequência

3.2.1 Contagem de operações no Shell Sort

A função `shell_sort_ops` desmembra o `while` composto em condições separadas para contar cada verificação:

```
1 def shell_sort_ops(arr):
2     ops = 0
3     n = len(arr)
4     gaps = [1, 4, 10, 23, 57, 132, 301, 701]
5     gaps = [g for g in gaps if g < n]
6     for gap in reversed(gaps):
7         for i in range(gap, n):
8             temp = arr[i]
9             ops += 1                       # atribuicao de temp
10            j = i
11            while True:
12                ops += 1                     # verificacao j >= gap
13                if j < gap:
14                    break
15                ops += 1                     # arr[j - gap] > temp
16                if not (arr[j - gap] > temp):
17                    break
18                arr[j] = arr[j - gap]
19                ops += 1                     # atribuicao no deslocamento
20                j -= gap
21            arr[j] = temp
22            ops += 1                         # atribuicao final
23    return arr, ops
```

A separação do `while` composto (`j >= gap and arr[j - gap] > temp`) em dois `if` consecutivos permite contar individualmente a verificação de índice e a comparação entre elementos, evitando o curto-circuito do `and` que esconderia operações.

4 Gráficos

4.1 Operações Totais vs N

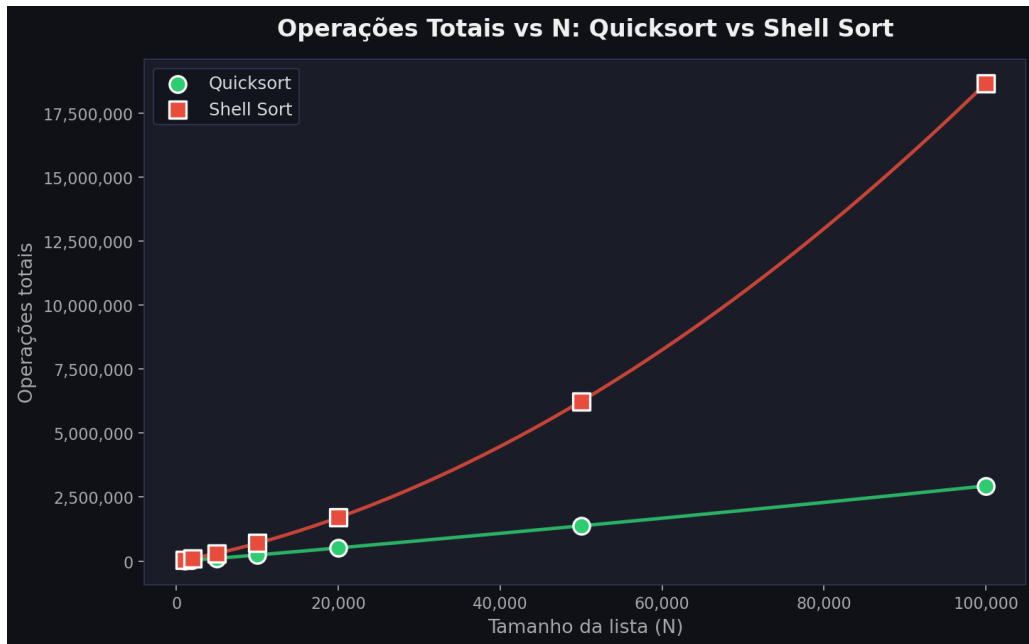


Figure 1: Operações vs N

O crescimento do número total de operações em função do tamanho da lista para Quicksort e Shell Sort. A escala linear evidencia que o Shell Sort realiza muito mais operações que o Quicksort para os mesmos N, especialmente à medida que N cresce.

O Quicksort apresenta crescimento $O(n \log n)$ visível na curvatura suave e amortecida. O Shell Sort exibe crescimento mais acelerado, consistente com sua complexidade teórica entre $O(n^{1.25})$ e $O(n^2)$, dependendo da sequência de gaps.

4.2 Comparação Normalizada — Quicksort

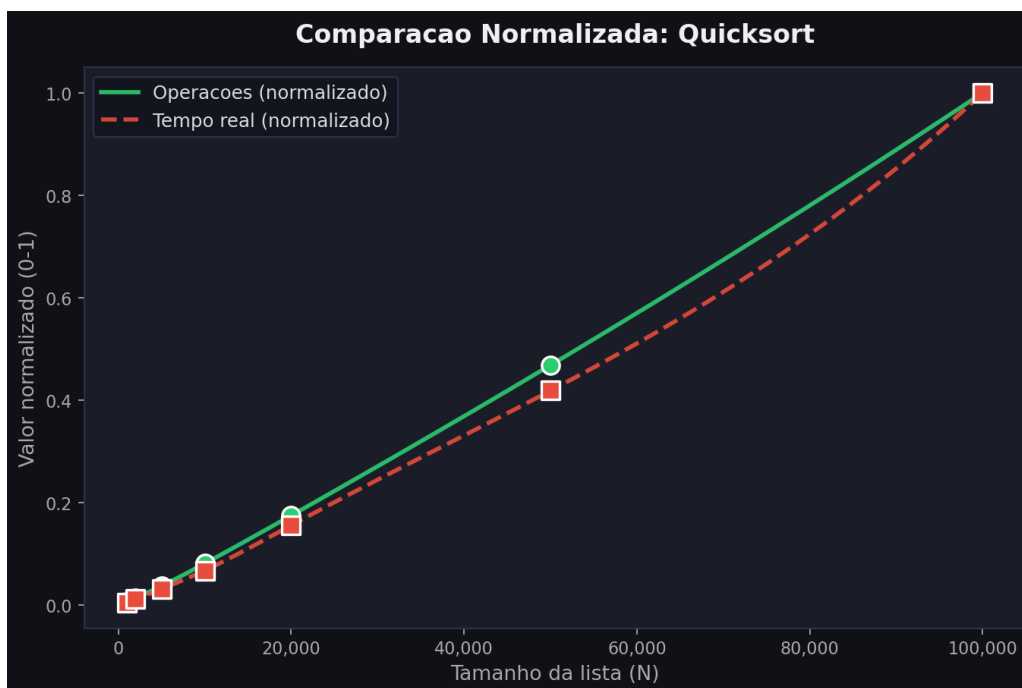


Figure 2: Comparação Normalizada Quicksort

As curvas praticamente coincidem, comprovando que a contagem de operações é diretamente proporcional ao tempo de execução real. O Quicksort mantém uma proporcionalidade quase perfeita porque suas operações são predominantemente comparações e swaps de custo constante.

4.3 Comparação Normalizada — Shell Sort

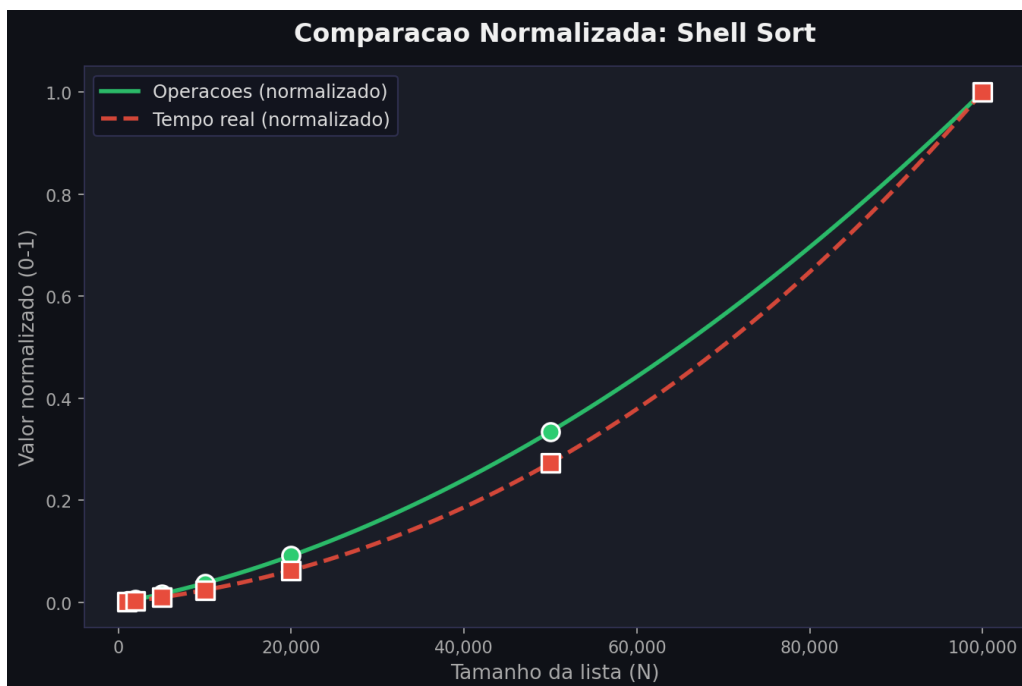


Figure 3: Comparação Normalizada Shell Sort

A sobreposição é boa, confirmando que mesmo para um algoritmo com padrão de acesso à memória menos regular (acessos com stride variável), a contagem de operações elementares captura corretamente o crescimento da complexidade.

5 Resultados

Table 1: Número total de operações por tamanho de entrada N

N	Quicksort (ops)	Shell Sort (ops)
1.000	18.281	46.759
2.000	40.379	105.338
5.000	110.054	304.028
10.000	239.214	699.708
20.000	514.334	1.703.781
50.000	1.376.549	6.243.406
100.000	2.937.978	18.660.359